

Assignment 1 Report

Static Analysis and Code Quality

STUDENT

Muhammad Aun Anis

ROLL NUMBER

24P-3017

PROJECT

Baccarat Punto Banco

ANALYSIS BRANCH

static-analysis

BASELINE

Original source on main

DATE

27 September 2026

Baseline findings, independent review, refactoring, and verification

1. Project and version scope

This report analyzes the six Python modules listed in the project README: `baccarat-cli.py`, `baccarat-sim.py`, `cards.py`, `hands.py`, `players.py`, and `rules.py`. The baseline is the original `main` source. All source refactoring described below is on `static-analysis`; `main` was not modified. The baseline Pylint output supplied with the project is retained in `pylint/pylint-baseline.txt`.

The supplied baseline records **9.17/10**. Its complete output contains **38 messages**: 22 convention messages, 7 refactor messages, 8 warning messages, and 1 error. The warning count includes three unused-code messages (two unused variables and one unused import). The largest counts are `rules.py` (10) and `cards.py` (9), followed by `baccarat-cli.py` (7) and `baccarat-sim.py` (6). Categories here describe the issues by quality concern; Pylint's single-letter severity groups are not treated as quality categories.

For a like-for-like rerun with the project configuration and installed Pylint 4.0.8, the original source scored **9.20/10 with 36 messages** and the modified source scored **10.00/10 with no messages**. The configured run suppresses `invalid-name` for the project's legacy hyphenated executable filenames. The baseline's original 9.17 score is preserved as recorded; the configured rerun is used for the comparable before/after figures.

2. Initial static-analysis evidence

The full supplied initial report is ``pylint/pylint-baseline.txt``. Its recorded score is 9.17/10. The findings, counted from that output, are:

Quality concern	Count	Representative messages
Documentation	10	missing module, function, or class docstring
Naming / file convention	2	hyphenated executable module names
Error handling / possible defect	4	bare <code>except</code> ; undefined <code>GameError</code>
Control-flow clarity	6	unnecessary <code>else / elif</code> after <code>return</code> or <code>raise</code>
Dead or unused code	3	unused loop variables and import
Formatting	4	trailing whitespace, indentation, long line
Function consistency	1	inconsistent returns in <code>Card.__repr__</code>
API / idiom	7	explicit <code>__str__</code> calls and dictionary key iteration
File portability	1	file opened without explicit encoding
Total	38	

Pylint is effective at finding these local, mechanically recognizable issues. The missing `GameError` import in the CLI is a direct defect: submitting a bet that triggers the exception handler can itself raise `NameError`.

3. Ten investigated Pylint findings

The following are ten distinct findings selected from the initial baseline. The code excerpts are from the original source; line numbers refer to the baseline Pylint output and may move after edits.

ID	Location	Pylint message / ID	Quality impact	Decision
P1	<code>baccarat-cli.py:111</code>	Undefined variable <code>GameError</code> / E0602	Reliability: the error path crashes while handling an invalid bet.	A - fix
P2	<code>baccarat-cli.py:60, 107, 186</code>	No exception type specified / W0702	Reliability and diagnosability: unexpected exceptions can be silently swallowed.	A - fix
P3	<code>baccarat-sim.py:39</code>	open without encoding / W1514	Portability: text output can depend on the host's default encoding.	A - fix

ID	Location	Pylint message / ID	Quality impact	Decision
P4	<code>cards.py:53</code>	Inconsistent return statements / R1710	Reliability and maintainability: a valid <code>Card</code> can produce <code>None</code> from <code>repr</code> for an unexpected rank representation.	A - fix
P5	<code>cards.py:112, 129</code>	Unused loop variable <code>i</code> / W0612	Readability: the loop index implies that position matters when it does not.	A - fix
P6	<code>hands.py:110</code>	Unnecessary <code>elif</code> after <code>return</code> / R1705	Readability: avoid an extra control-flow branch after an unconditional return.	A - fix
P7	<code>players.py:133</code>	Missing class docstring / C0115	Maintainability: callers lack a description of the custom settlement exception.	A - fix
P8	<code>rules.py:61, 97, 161, 164, 167</code>	Unnecessary <code>__str__</code> calls / C2801	Readability and maintainability: direct dunder calls bypass the normal conversion idiom.	A - fix
P9	<code>baccarat-sim.py:74, 80</code>	Consider iterating with <code>.items()</code> / C0206	Readability: separate key lookup obscures the key/value relationship.	A - fix
P10	<code>baccarat-cli.py:1</code>	Missing module docstring / C0114	Maintainability: module purpose is not discoverable from the source itself.	A - fix

Examples of original findings and interpretation

P1 - undefined exception (`baccarat-cli.py`, original line 111)

```
except (ValueError, TypeError, GameError) as error:
```

The CLI refers to `GameError` without importing it. Python resolves exception names when the handler is reached, so an invalid wager that raises a `ValueError` can be replaced by a `NameError` while checking whether it matches this handler. Importing the actual exception type makes the error path executable and testable.

P2 - bare exception handling (`baccarat-cli.py`, original lines 60, 107, 186)

```
try:
    balance_input = int(balance_input)
except:
    pass
```

This catches exceptions unrelated to bad user input and leaves the unconverted value for a later operation to reject. It also hides which operation failed. The CLI now converts input explicitly and catches only expected validation exceptions.

P3 - implicit output encoding (`baccarat-sim.py`, original line 39)

```
with open(file_name, 'w') as sim_file:
```

The generated report is text, but the encoding was inherited from the machine. Explicit UTF-8 makes the output behavior predictable across environments.

P4 - incomplete `repr` return (`cards.py`, original lines 53-58)

```
if isinstance(self._rank, str):
    return f'Card(\'{self._rank}\', \'{self._suit}\')'
elif isinstance(self._rank, int):
    return f'Card({self._rank}, \'{self._suit}\')'
```

The constructor currently restricts ranks to strings or integers, but `repr` has no fallback return. A single representation using `!r` covers both valid forms and ensures the method always returns a string for valid cards.

P5 - unused loop index (`cards.py`, original lines 112 and 129)

```
for i in range(num_decks):
    ...
```

The index is never used. `_` communicates that only repetition matters and avoids suggesting an omitted positional operation.

P6 - redundant branch (`hands.py`, original line 110)

```

if 0 <= self.value <= 2:
    return True
elif 3 <= self.value <= 6:
    ...

```

After the first return, the second condition does not need an `elif`. This is a readability improvement and does not change the banker draw rule.

P7 - undocumented exception (`players.py`, original line 133)

```

class InvalidBet(Exception):
    pass

```

This exception is part of the player settlement API. A short class docstring explains when callers can expect it; the class already receives that documentation in the modified source.

P8 - direct dunder invocation (`rules.py`, original lines 61, 97, 161, 164, 167)

```

return ', '.join([card.__str__() for card in self._punto.cards])

```

Calling `str(card)` expresses the intended conversion through Python's normal protocol and removes unnecessary dunder calls. It also permits idiomatic implementation changes to the object's string conversion.

P9 - dictionary key lookup (`baccarat-sim.py`, original lines 74 and 80)

```

for win in shoe_wins:
    sim_file.write(f'{win.title()}: \t{shoe_wins[win]}\n')

```

The loop immediately looks up each value by key. Iterating over `.items()` makes the pair explicit and avoids repeated dictionary access.

P10 - no module purpose (`baccarat-cli.py`, original line 1)

```

import time

```

The module docstring now identifies the CLI's role. This helps readers distinguish the interactive entry point from the simulation module and game-rule modules.

Additional baseline finding considered context-dependent

The baseline also reports C0103 for `baccarat-cli.py` and `baccarat-sim.py`. The hyphenated names match the existing documented command (`python baccarat-cli.py`) and are therefore a project-interface choice, not a naming mistake that can be corrected without changing how users launch the scripts. The project configuration disables `invalid-name` for this repository and documents why. Pylint's default recommendation is not universally appropriate here.

4. Independent manual code review

These findings came from reading behavior and state transitions rather than copying the Pylint list. Findings M1 and M2 are not directly reported by Pylint.

ID	Original code / location	Problem and quality impact	Recommended improvement / status
M1*	<code>rules.py: if not self._game_running: ...; if self._punto.value > self._banco.value:</code>	Before any deal, <code>_game_running</code> is false while both hands are None; the method leaks <code>AttributeError</code> instead of the API's <code>GameError</code> . Reliability and diagnostic clarity are affected.	Check that both hands exist and raise <code>GameError</code> ; implemented and covered by a regression check.
M2*	<code>cards.py: if not num_decks: num_decks = self._num_decks</code>	An explicit zero is treated as "use the default," hiding invalid caller input. This weakens validation and makes configuration mistakes hard to detect.	Use <code>is None</code> for the default and validate the explicit value; implemented.
M3	<code>baccarat-cli.py: except (ValueError, TypeError) as error: ...; self.add_player()</code>	Repeated mistakes grow the Python call stack and complicate cancellation. Reliability and testability are affected.	Use a loop or return to the menu after an invalid entry; player and shoe setup now use loops / non-recursive error handling.

ID	Original code / location	Problem and quality impact	Recommended improvement / status
M4	rules.py: for player in self._players: ... self._players.index(player)	This performs repeated linear searches and obscures the association between a player and its index. Performance is minor for this app; readability and maintainability are the real concerns.	Use enumerate; implemented.
M5	hands.py: third_card_rules = {3: [...], 4: [...], 5: [...], 6: [...]} inside Banco.draw_third	The canonical rule is difficult to compare as a whole against a game-rules reference; changes risk an overlooked branch. Testability and modifiability are affected.	Name the rule table and test representative banker totals and player third-card values. This refactor remains recommended work; the original decision table is preserved.

*Not directly reported by Pylint.

5. Pylint configuration choices

The project uses `.pylintrc`. Three selected decisions are:

Setting	Default / current choice	Decision and reason
max-line-length	Pylint default is 100; configured as 100.	Retain. The six modules are compact and 100 columns keeps code review readable. The one overlong baseline line is a real formatting issue, not a reason to relax the project standard.
max-args	Default is 5; configured as 5.	Retain. Public game operations should remain understandable without accepting long parameter lists. No current finding justifies raising the limit.
max-branches	Default is 12; configured as 12.	Retain. Rule-heavy code can need branching, but the current limit gives a useful signal without forcing a change to the Baccarat rules.
invalid-name	Enabled by default; disabled for this project.	Change. The two hyphenated executable names are already the project's user-facing launch commands. Renaming them would alter that interface solely to satisfy a module-name convention.
py-version	Set to 3.10.	Set explicitly to the project's supported syntax/runtime floor; this makes version-sensitive checks reproducible.

6. Improvements made

At least five significant improvements were selected; several are structural or behavioral quality changes rather than cosmetic corrections.

11 - Repair CLI exception handling and avoid recursive retries

Original: the CLI named `GameError` without importing it; it used bare `except` while attempting conversions and recursively re-entered handlers after bad input.

```
try:
    amount_input = int(amount_input)
except:
    pass
self._game.bet(player_i, hands.get(hand_input.lower()), amount_input)
```

Improved: import `GameError`, convert input once, reject unknown bet labels explicitly, catch only expected validation exceptions, and avoid recursive calls. Player creation and shoe setup retry through bounded loops.

```
amount = int(amount_input)
hand = hands.get(hand_input.lower())
if hand is None:
    raise ValueError('Select punto, banco, or tie.')
self._game.bet(player_i, hand, amount)
```

This makes failures predictable and prevents a user-input loop from consuming call stack. An invalid wager no longer triggers a secondary `NameError`.

I2 - Make shoe defaults and card draws validate explicitly

Original: `if not num_decks` silently substituted the default even when callers explicitly supplied zero; draw loops accepted nonsensical counts.

```
if not num_decks:
    num_decks = self._num_decks
```

Improved: `None` alone selects the default. Deck and card counts are checked for type and valid range before state changes. This improves reliability and makes invalid API calls fail close to their source.

```
if num_decks is None:
    num_decks = self._num_decks
if not isinstance(num_decks, int):
    raise TypeError('Number of decks must be an integer.')
if num_decks < 1:
    raise ValueError('Number of decks must be positive.')
```

I3 - Guard game-result state and use explicit game errors

Original: `game_result()` checked `_game_running`, but before the first deal it then dereferenced two `None` hands and raised an implementation-level `AttributeError`.

```
if self._game_running:
    raise GameError('Game is running.')
if self._punto.value > self._banco.value:
    return 'punto'
```

Improved: the method now checks for missing hands and raises `GameError('No hands have been dealt.')`. This preserves the domain API and gives callers a stable error to handle. A regression test covers the pre-deal call.

```
if self._punto is None or self._banco is None:
    raise GameError('No hands have been dealt.')
```

I4 - Replace payout magic values with named integer ratios

Original: `win()` embedded 1, 0.95, and 8 in payout branches.

```
self._balance += int(self._amount_bet * 0.95)
```

Improved: named constants describe the punto, banco, and tie payout rules. Banco payout uses integer numerator/denominator arithmetic, preserving the prior truncation for positive bets without a floating-point intermediate. The values are now easier to review against the rule and change consistently.

```
BANCO_PAYOUT_NUMERATOR = 95
BANCO_PAYOUT_DENOMINATOR = 100
self._balance += (self._amount_bet * BANCO_PAYOUT_NUMERATOR
                  // BANCO_PAYOUT_DENOMINATOR)
```

I5 - Remove redundant player lookup

Original: `available_players` iterated over `players` and then called `self._players.index(player)` to rediscover each index.

```
for player in self._players:
    if player.balance > 0:
        players.append(self._players.index(player))
```

Improved: iteration uses `enumerate(self._players)`. This is linear rather than repeatedly scanning the list and directly expresses the index/player relationship.

```
for index, player in enumerate(self._players):
    if player.balance > 0:
        players.append(index)
```

I6 - Make simulation output portable and its summaries clearer

Original: output used the platform default encoding and iterated dictionary keys before looking up each count.

```
with open(file_name, 'w') as sim_file:
    for win in total_wins:
        sim_file.write(f'{win.title()}: \t{total_wins[win]}\n')
```

Improved: the file is opened with UTF-8, and summary loops use `.items()` with a named count; percentage calculation is assigned a descriptive variable. This improves portability and makes the output code easier to inspect.

```
with open(file_name, 'w', encoding='utf-8') as sim_file:
    for winner, count in total_wins.items():
        percentage = round((count / game_count) * 100, 4)
        sim_file.write(f'{winner.title()}: \t{count} \t({percentage}%)\n')
```

The simulation's total-summary writing and timestamped filename generation were also extracted into small helpers. This reduces local state in `main()` and separates summary formatting from simulation control flow.

17 - Complete simple representations and string conversions

`Card.__repr__` now returns one consistent expression using `!r`, loop-only variables are `_`, and card display uses `str(card)`. These changes improve predictability and remove dead-code/idiom findings without changing card values or displayed wording.

7. Verification evidence

There were no automated tests in the original project. A focused `unittest` suite was added at `tests/test_quality_refactor.py`. It checks card values and representation, one-deck shoe size and draws, invalid counts, pre-deal result errors, stable table player indexes, and payout/balance behavior.

Command: `python -m unittest discover -s tests -v`. Result: **7 tests passed**; the captured output is `pylint/unittest-results.txt`. A CLI smoke run accepted `0` then `y` and exited cleanly (captured output). A simulation smoke run (`python ../baccarat-sim.py -s 1 -d 1`, from a temporary working directory) completed one shoe and created its timestamped text report (captured output).

The focused checks cover the changed domain behavior. The application entry points and interactive error paths were not exhaustively exercised; no claim of exhaustive game-rule coverage is made. The game uses random shuffling, so these unit checks avoid asserting a particular draw order.

8. Final Pylint results

The full modified-source output is `pylint/pylint-after.txt`. The configured baseline rerun is `pylint/pylint-baseline-configured.txt`. Both use Pylint 4.0.8 and the same `.pylintrc`.

Metric	Original, configured	Modified, configured	Change
Pylint score	9.20/10	10.00/10	+0.80
Total findings	36	0	-36
Convention issues (C)	20	0	-20
Warnings (W)	8	0	-8
Refactoring findings (R)	7	0	-7
Errors (E/F)	1	0	-1

Pylint's `W` group includes the baseline unused import and unused variables, so the warning reduction includes those three dead-code findings. Convention findings improved the most, including module/function/class docs, output idioms, dictionary iteration, whitespace, and indentation. The final configured run reports no findings. The 10.00 score reflects this configured rule set and does not establish complete correctness or design quality.

The score improvement is consistent with fewer static-analysis findings, but it is not a complete measure of design quality. In particular, the manual pre-deal state defect was invisible to Pylint. The independent regression check found and verified a behavior issue despite the initial score already being 9.17/10.

9. Static analysis and human review

1. **What did Pylint detect well?** Mechanical source issues: missing documentation, unused names/imports, unsafe broad exception handling, inconsistent return paths, explicit dunder calls, implicit encoding, and simple control-flow/style problems.
2. **What needed human judgment?** Whether a method is responsible for too much, whether a nested rule is understandable and correct, whether an exception type is meaningful, and whether a reported naming convention conflicts with the established CLI interface. The pre-deal `AttributeError` and `Shoe.add_decks(0)` defaulting defect required tracing state and input semantics.
3. **Any false positive or issue not worth fixing?** The hyphenated script-name messages are acceptable in this project because they are existing documented entry points. The configuration records that exception rather than renaming the interface. The banker draw table is not inherently wrong simply because it is a mapping; a future change should preserve verified Baccarat rules.
4. **Can a high score coexist with poor design?** Yes. The baseline scored 9.17/10, yet `game_result()` raised an incidental `AttributeError` before the first deal, and `Shoe.add_decks(0)` silently selected a default. Pylint did not report either behavior.
5. **Should static analysis replace human review?** It should complement review. It quickly finds repeatable local issues and guards conventions, while human review evaluates domain rules, state transitions, design trade-offs, and whether a tool suggestion fits the product interface.